Adaptive switching — detailed description

High-level idea

Adaptive switching is an automated policy layer that continuously monitors device/server *metrics* (telemetry, integrity checks, environment signals) and, when one or more *heuristics* detect an anomalous condition, proposes switching the encryption pipeline to a different **recipe** (a predefined ordered list of algorithms). The goal is to increase resistance under attack while minimizing unnecessary overhead during benign operation.

Key properties:

- **Only one recipe is active** at a time (global state). ZTM enables automatic switching; in normal mode, the system only raises alerts.
- Heuristics run independently and can each propose a target recipe when their condition is met.
- An **arbiter** (AdaptiveSwitchboard) collects proposals, enforces rate limits and cooldowns, resolves conflicts, and performs an **atomic** transition so both sender and receiver remain synchronized (preventing decryption failure).
- Strict **master key & nonce** discipline must be preserved across the switch. Never reuse nonces; perform switch at packet boundaries and confirm sync.

Inputs / metrics used to decide a switch (typical list)

Each heuristic uses one or more metrics. Common ones used in XenoCipher:

- **CPU usage** (percent): sudden spike may indicate crypto-probing or stress.
- **Heap / free RAM**: memory exhaustion attempts or runaway allocations.
- **HMAC failure rate**: integrity failures or active tampering.
- **Replay / nonce reuse rate**: evidence of replay attack or device compromise.
- **Packet loss / retransmit count**: network anomalies, flooding, or MITM.
- **Throughput / latency anomalies**: large deviations from baseline.
- **Entropy / randomness drop**: poor RNG or tampering with keystream.
- **External signals**: e.g., suspicious IP / flow patterns reported by server, or high-confidence IDS alerts.

Each heuristic entry in `heuristics.json` contains:

- `metric_name` and threshold (e.g., CPU > 85%)
- `direction` (above/below)
- `target_recipe` (recipe to switch to)
- `CD` (cooldown seconds)
- `MaxS/h` (max switches per hour)
- `Conf` (minimum aggregated confidence required)
- `priority` (optional, for conflict resolution)

How each heuristic forms a proposal

1. Read current metric(s).
2. Compute a **normalized score/confidence** (0..1) versus baseline and thresholds (e.g., via z-score or min-max).
3. Smooth the score (e.g., EMA) to avoid twitchy responses.
4. If `score >= Conf` and rate-limit/cooldown allow, emit a `recipe_proposal = (target_recipe, score, heuristic_id)` into AdaptiveSwitchboard.

> Important: heuristics are **independent** proposers — they do not immediately change the active recipe; they only propose.

Arbitration / decision logic (AdaptiveSwitchboard)

AdaptiveSwitchboard receives proposals and executes the following (summary):

1. **Collect proposals** within a decision window (e.g., 1 second).
2. **Filter** proposals that violate per-heuristic `CD` (cooldown) or `MaxS/h`.
3. **Aggregate** proposals by recipe: compute aggregated confidence for each candidate recipe (e.g., weighted sum or max of scores).
4. **Apply hysteresis / priority**:
    - Require aggregated confidence ≥ recipe's `min_conf` (or general `Conf`) AND
    - If the candidate is different from the active recipe, verify that switching limits allow it.
    - If multiple candidates tie, choose highest `priority` or highest aggregated confidence.
5. **Safety checks**:
    - Ensure last switch time + global cooldown passed.
    - Ensure active transmissions are at a safe boundary (drain buffers or wait for next packet boundary).
    - Ensure both endpoints can derive the per-recipe subkeys (HKDF uses same master key + next nonce).
6. **Perform atomic switch** (detailed below).
7. **Log and broadcast** the switch (EventBus / WebSocket) and apply per-heuristic cooldown resets as needed.

Safe, atomic switch procedure (sender + server)

A safe transition is critical to avoid decryption errors. The general sequence:

1. **Prepare switch**:
    - AdaptiveSwitchboard chooses `newRecipe`.
    - Compute the *first nonce* that will be used under the new recipe (usually the next per-packet nonce).
    - Derive per-recipe subkeys for the upcoming packet(s) from the same `master_key` + planned nonce, so both sides can precompute if needed.
2. **Notify peer**:
    - Send a `recipe_switch_intent` event to server (or device) containing:
        - `newRecipe` name
        - `effective_nonce` (the nonce where the switch becomes active)
        - `timestamp` and `switch_id` (unique)
        - optionally `signature/HMAC` to authenticate the switch command
3. **Wait for ACK**:
    - Peer validates request, ensures it can derive subkeys, responds `ACK` (or `NACK`).
    - If NACK, abort and log.
4. **Switch at boundary**:
    - Both sides commit the switch only when packet counter (nonce) reaches `effective_nonce`. Implementation typically:
        - Finish current packet(s) using old recipe.
        - Increment nonce to `effective_nonce`.
        - Start encrypting/decrypting new packets using `newRecipe`.
5. **Commit and log**:
    - AdaptiveSwitchboard marks `lastSwitchTime` and increments per-hour counter.
    - Broadcast `recipe_switch` event to clients (UI updates).
6. **Rollback handling**:
    - If excessive decryption failures occur immediately after the switch, a rollback policy can be triggered: revert to previous recipe or signal operator. (Use sparingly — this must be guarded by checks to avoid oscillation.)

How recipes are proposed and switched independently of the active recipe

- Each heuristic acts on *raw metrics* and has its own `target_recipe`. They do not need to know which recipe is active; they simply propose.
- AdaptiveSwitchboard performs the arbitration: proposals are aggregated and mapped to concrete transitions.
- Example: Suppose `CPU_spike_heuristic` proposes `CHA_CHA_HEAVY` and `HMAC_failure_heuristic` proposes `FULL_STACK`. The arbiter may decide `FULL_STACK` has higher aggregated confidence or priority and select it.
- Because proposals are metric-driven and independent, a heuristic that targets `SALSA_LIGHT` can still propose `SALSA_LIGHT` even if `SALSA_LIGHT` is not active. If it is already active, the arbiter will ignore redundant proposals (no-op).

Key point: **only the arbiter changes the active recipe** after satisfying cooldowns, rate limits and safety checks. This prevents multiple heuristics from fighting to directly mutate the pipeline.

Configuration knobs to prevent oscillation and unsafe behavior

- **Cooldown (CD)**: a per-heuristic cooldown/time-to-wait after a switch before the heuristic may trigger another switch. Prevents bouncing.
- **Max switches/hour**: rate-limits total recipe changes to avoid runaway CPU usage.
- **Confidence aggregator & hysteresis**: require a higher threshold to switch back to a lighter recipe than to switch up (avoid flip-flops).
- **Priority levels**: allow critical heuristics (e.g., mass HMAC failures) to override benign ones (e.g., occasional latency).
- **Decision window / debounce**: aggregate proposals over a short window (e.g., 1–3 s) before deciding.
- **Manual override**: user-initiated manual switch has the highest priority and can optionally suspend automatic switching until re-enabled.

Master key & nonce safety during switching

- **Never re-use nonces**. Every packet must have a unique nonce for HMAC/key derivation. During a switch, choose an explicit `effective_nonce` and ensure both ends adopt it atomically.
- **Master key remains unchanged** during recipe switches (it is the root secret). All recipes derive per-packet subkeys from the *same* master key + nonce using HKDF. This allows both sides to create keys deterministically without a large handshake on every switch.
- **Subkey derivation**: per-packet subkeys = `HKDF(master_key, nonce, label)` — this produces LFSR seeds, Tinkerbell keys, transposition keys, etc. Because HKDF inputs include the nonce, keys are unique per packet and per recipe usage.
- **Switch handshake** must convey the effective nonce and be authenticated (HMAC or signed) to prevent an attacker from spoofing a recipe switch.
- **Atomic boundary**: Only change recipe at the agreed nonce boundary. This ensures sender and receiver derive the same keys for a packet.

Example decision pseudocode (compact)

```
loop every DECISION_INTERVAL:
    proposals = collect_proposals_window()   # list of (recipe, score, heuristic_id)
    filtered = filter_by_cooldown_and_rate(proposals)
    aggregated = aggregate_by_recipe(filtered)  # recipe -> aggregate_score

    candidate = select_best(aggregated)  # apply priority & confidence thresholds
    if candidate and candidate.recipe != active_recipe:
        if passes_global_rate_limits() and candidate.score >= MIN_CONF:
            effective_nonce = next_nonce() + SWITCH_DELAY
            send_switch_intent(peer, candidate.recipe, effective_nonce)
            if wait_for_ack(timeout):
                commit_switch_at_nonce(effective_nonce, candidate.recipe)
            else:
                log_warn("switch NACK or timeout -> abort")
```

Example scenario walk-through

- **Initial**: ACTIVE = `SALSA_LIGHT` (LFSR + Salsa20), nonces in sync.
- **Event**: HMAC failure rate rises above threshold. `hmac_heuristic` proposes `FULL_STACK` with score 0.92.
- **Arbiter**: sees proposal, no cooldown violation, aggregated score meets `Conf`. Global rate limits permit.
- **Switch**: Arbiter computes `effective_nonce = current_nonce + 2`, sends `recipe_switch_intent` (signed) to server. Server ACKs.
- **Boundary**: two packets are finished under `SALSA_LIGHT`. When `nonce == effective_nonce`, both ends start using `FULL_STACK`. HKDF with that nonce yields per-packet subkeys for all algorithms in `FULL_STACK`. No nonce reuse. Switch logged and UI updated.

Practical recommendations / safety checks to implement

1. **Atomic switching at packet boundary** — never mid-packet.
2. **Authenticated switch messages** — sign or HMAC switch commands using the session HMAC key.
3. **Precompute/verify subkey sizes** for all recipes so both peers can allocate memory ahead of time.
4. **Graceful rollback** policy if decryption error spikes after a switch; but make rollback require human confirmation or very strong evidence to avoid attacks that force oscillation.
5. **Extensive logging & telemetry** — record every proposal, aggregated score, arbitration decision, and effective nonce. These logs are useful for debugging and for the UI to explain why a recipe change happened.
6. **Test corner cases**:
    - What happens if peer ACK is lost? (Use retries and fail open/closed policy.)
    - What if effective_nonce is too close (no time to drain buffers)? (Enforce minimum switch delay.)
7. **UI visibility** — show proposed reasons, aggregated confidence, and cooldown timers so operator can understand and optionally override.

Short checklist: what to implement / verify in code

- Heuristic producers: compute normalized score + smoothing.
- Proposal queue + decision window.
- Aggregation logic + priority/hysteresis rules.
- Rate-limit counters (`MaxS/h`) and per-heuristic cooldowns (`CD`).
- Authenticated `recipe_switch_intent` + ACK handshake.
- Atomic boundary commit using `effective_nonce`.
- HKDF derivation uses `master_key + nonce` consistently across recipes.
- Logging / EventBus / WebSocket notifications for UI.